Практическая работа №4

Тема: разработка, обеспечение безопасности и обновление корпоративной блокчейн-сети Hyperledger Fabric

Цель: получить практические навыки проектирования корпоративной - блокчейн-сети Hyperledger Fabric, разработки Chaincode, управления идентификацией и конфиденциальностью, анализа угроз, проверки бизнес-логики, а также безопасного обновления Chaincode и компонентов блокчейн-сети.

Теоретическая часть.

Hyperledger Fabric - корпоративная платформа распределённого реестра с контролируемым членством участников. В отличие от публичных блокчейнов, доступ к сети предоставляется идентифицированным пользователям и организациям, а их полномочия определяются политиками безопасности. Архитектура Fabric разделяет функции идентификации, выполнения бизнес-логики, подтверждения и упорядочивания транзакций, а также хранения данных.

Основными компонентами являются:

- Client Application,
- Fabric Gateway,
- Peer,
- Ordering Service,
- Ledger.

Для управления цифровыми идентификаторами используются Certificate Authority и Membership Service Provider (MSP), а бизнес-логика реализуется посредством Chaincode.

Обработка транзакций в Hyperledger Fabric осуществляется по модели

Execute → Order → Validate → Commit**.**

Сначала Peer выполняют симуляцию Chaincode и формируют Read Set и Write Set без непосредственного изменения Ledger. После получения необходимых подтверждений транзакция передаётся Ordering Service для упорядочивания и включения в блок. Перед фиксацией Peer проверяет Endorsement Policy и актуальность прочитанных данных с помощью MVCC. Если состояние изменилось после симуляции, транзакция признаётся недействительной и её изменения не применяются к World State. Если версия ключа, включённого в Read Set транзакции, не соответствует актуальной версии в World State на этапе валидации, транзакция признаётся недействительной». Это точнее описывает механизм MVCC в Hyperledger Fabric.

Управление доверием осуществляется с помощью политик подтверждения и механизмов авторизации. Endorsement Policy определяет, какие организации должны подтвердить результат выполнения Chaincode, тогда как сам Chaincode контролирует полномочия конкретного пользователя на выполнение бизнес-операции. При этом необходимо учитывать идентичность участника, его организацию и роль, текущее состояние объекта и установленные правила бизнес-процесса.

Для разграничения информации Fabric использует Channels и Private Data Collections (PDC)**.** Channels изолируют отдельные группы участников и их реестры, а PDC позволяют ограничить доступ только к определённым конфиденциальным данным внутри общего Channel.

Безопасность Fabric зависит не только от корректности Chaincode, но и от архитектуры сети, управления идентичностями, политик доступа и бизнес-логики. Анализ безопасности включает определение активов, участников, доверительных границ, угроз и возможных последствий.

Для моделирования угроз могут применяться STRIDE, PASTA и DREAD, а при разработке Chaincode особое внимание уделяется нарушениям авторизации, некорректным переходам состояния, повторному выполнению операций и нарушениям бизнес-инвариантов. Ошибка бизнес-логики может

возникнуть даже при технически корректном выполнении программы, если правила системы допускают недопустимое действие или состояние.

Обновление Hyperledger Fabric выполняется контролируемо и предполагает предварительное резервное копирование Ledger и MSP, последовательное обновление Ordering Service и Peer и последующее изменение Channel Capabilities**.** При переходе с Fabric 2.x на 3.x необходимо учитывать изменения совместимости и удаление устаревших механизмов. Capability V3_0 должна активироваться только после обновления соответствующих компонентов сети.

Chaincode обновляется отдельно через жизненный цикл:

Package → Install → Approve → Check Readiness → Commit.

При изменении определения Chaincode увеличивается sequence, а новая реализация должна сохранять совместимость с состоянием, созданным предыдущей версией.

Для Ordering Service Hyperledger Fabric поддерживает различные модели отказоустойчивости. Raft обеспечивает Crash Fault Tolerance (CFT) и предназначен для сохранения работы сети при отказе части узлов. SmartBFT, доступный в Fabric 3.x, реализует Byzantine Fault Tolerance (BFT) и учитывает возможность некорректного или злонамеренного поведения части ordering nodes. Для BFT используется принцип 3f + 1, где f обозначает число византийских отказов, которые система должна выдержать. Выбор между Raft и SmartBFT определяется моделью доверия, требованиями к безопасности и необходимым уровнем отказоустойчивости корпоративной сети.

В качестве рабочей среды используется операционная система Linux либо Windows Subsystem for Linux (WSL) с установленными Docker, Go и компонентами Hyperledger Fabric версии 3.1.x. Разработка и развёртывание смарт-контрактов (Chaincode) выполняются на языке Go. Перед выполнением команд необходимо установить Fabric binaries, загрузить Docker-образы и настроить соответствующие переменные окружения.

https://hyperledger-fabric.readthedocs.io/en/release-2.5

Задание 1. Разработка корпоративной блокчейн-сети Hyperledger Fabric

1. Создать корпоративную блокчейн-сеть Hyperledger Fabric, включающую две организации, Peer-узлы, Ordering Service, Fabric CA и отдельный Channel.

2. Настроить среду командной строки Fabric и административную идентичность Org1 для последующей работы с Peer и Ordering Service.

3. Разработать архитектурную схему сети и Chaincode предметной области своего варианта. В Chaincode реализовать не менее шести операций: создание объекта, чтение, изменение состояния, основную бизнес-операцию, проверку существования и получение списка объектов. Предусмотреть валидацию входных данных и допустимых переходов состояния.

4. Развернуть разработанный Chaincode с version = 1.0, sequence = 1 и Endorsement Policy, требующей подтверждения обеих организаций. Проверить зафиксированное определение Chaincode.

5. Выполнить не менее четырёх транзакций с разработанным Chaincode и зафиксировать их TxID. Одну транзакцию проанализировать по этапам Proposal → Endorsement → Ordering → Validation → Commit.

6. Проверить текущее состояние бизнес-объектов после выполнения транзакций.

7. Исследовать Ledger созданного Channel, определить высоту Blockchain и получить последний блок. Объяснить различие между Blockchain и World State.

Задание 2. Разработка механизмов безопасности Chaincode и конфиденциальности данных

1. Разработать модель угроз корпоративной блокчейн-системы, определив активы, участников, доверительные границы, угрозы, риски и меры защиты. Используя STRIDE, определить не менее шести угроз для Client, Gateway, Peer, Chaincode, Ordering Service, MSP и Ledger.

2. Создать контролируемо уязвимую версию Chaincode с одной ошибкой бизнес-логики: отсутствием авторизации, недопустимым переходом состояния, повторным выполнением операции либо нарушением уникальности объекта. Развернуть её под отдельным именем, чтобы не изменять sequence основного Chaincode.

3. Воспроизвести уязвимость от имени Org2, зафиксировать результат несанкционированной операции и определить нарушенный бизнес-инвариант.

4. Разработать защищённую версию Chaincode, реализовав проверку MSP, идентичности вызывающей стороны, полномочий пользователя, допустимых переходов состояния и бизнес-инвариантов. Для конфиденциальных данных реализовать работу через GetTransient(), PutPrivateData() и GetPrivateData().

5. Создать Private Data Collection, доступную только разрешённой организации, и развернуть защищённый Chaincode под отдельным именем.

6. Передать конфиденциальные данные в Chaincode через transient field и подтвердить, что исходное значение конфиденциальных данных не включается в обычные аргументы транзакции и не сохраняется в публичной части Ledger.

7. Проверить разграничение доступа и выполнить не менее шести security-тестов. Org1 должна иметь доступ к privateDetails, а запрос от Org2 должен завершаться отказом.

Задание 3. Обновление Chaincode и исследование отказоустойчивости Hyperledger Fabric

1. Зафиксировать исходное состояние основного Chaincode перед обновлением: version, sequence, Package ID и состояние ранее созданных бизнес-объектов. До обновления основной Chaincode должен иметь Version: 1.0 и Sequence: 1.

2. Разработать Chaincode версии 2.0, добавив новую функциональность или изменив модель бизнес-объекта с сохранением совместимости с ранее записанным World State. Создать новый Chaincode package и получить Package ID.

3. Установить Chaincode 2.0 на Peer обеих организаций.

4. Утвердить новое определение Chaincode обеими организациями, установив version = 2.0 и sequence = 2 и сохранив исходную Endorsement Policy.

5. Проверить согласование определения и зафиксировать Chaincode 2.0 в Channel. После Commit подтвердить значения Version: 2.0 и Sequence: 2.

6. Проверить обратную совместимость Chaincode 2.0: прочитать объект, созданный версией 1.0, выполнить старую операцию и проверить новую функцию версии 2.0. Дополнительно разработать план обновления инфраструктуры Fabric 2.5 → 3.1.x по последовательности Backup → Orderers → Peers → Verification → Channel Capabilities.

7. Создать отдельный стенд Fabric 3.x с SmartBFT Ordering Service и проверить сохранение работоспособности после контролируемого отказа одного Orderer. После остановки одного Orderer выполнить контрольную транзакцию, затем восстановить узел.

> *Остановка одного контейнера демонстрирует устойчивость к crash failure, но сама по себе не является полноценным доказательством устойчивости к произвольному Byzantine behavior.*

Варианты практической работы

№	Предметная область	Организация 1	Организация 2	Бизнес-объект	Конфиденциальные данные
1	Банковские операции	Банк-отправитель	Банк-получатель	PaymentOrder	Расчётные реквизиты
2	Страхование	Страховая компания	Экспертная организация	Claim	Персональные документы
3	Логистика	Отправитель	Перевозчик	Shipment	Коммерческие условия
4	Здравоохранение	Медицинская организация	Лаборатория	MedicalRecord	Медицинские данные
5	Образование	Университет	Проверяющая организация	Credential	Данные выпускника
6	Энергетика	Энергетическая компания	Оператор	MeterRecord	Тарифные данные
7	Нефтегазовая отрасль	Оператор	Сервисная компания	EquipmentRecord	Диагностические данные
8	Промышленность	Производственное предприятие	Сервисная компания	MachineService	Стоимость обслуживания
9	Телекоммуникации	Оператор связи	Подрядчик	NetworkService	Техническая конфигурация
10	Розничная торговля	Ритейлер	Поставщик	PurchaseOrder	Закупочная стоимость
11	Недвижимость	Продавец	Регистрационная организация	PropertyDeal	Документы сделки
12	Государственные услуги	Государственный орган	Сервисная организация	ServiceRequest	Персональные документы
13	Транспорт	Транспортная компания	Оператор	TransportRecord	Данные пассажира
14	Туризм	Туристическая компания	Поставщик услуги	Booking	Персональные данные клиента
15	Фармацевтика	Производитель	Дистрибьютор	MedicineBatch	Результаты контроля качества
16	Кибербезопасность	SOC	Организация-клиент	SecurityIncident	Цифровые доказательства
17	Облачные технологии	Cloud Provider	Организация-клиент	CloudResource	Конфигурационные данные
18	Сельское хозяйство	Производитель	Переработчик	ProductLot	Коммерческие данные
19	Спортивные системы	Спортивная организация	Аналитический центр	AthleteRecord	Персональные показатели
20	Экология и ESG	Предприятие	Аудитор	ESGRecord	Исходные данные аудита

Работа не может получить более **50 баллов**, если сеть или Chaincode не запускаются.
Работа не может получить более **70 баллов**, если отсутствует практическое доказательство разграничения доступа и исправления уязвимой бизнес-логики.
Работа не может получить более **80 баллов**, если не выполнено обновление Chaincode sequence 1 - 2 с доказательством сохранения старого состояния.
Публикация закрытых ключей, сертификатов с приватными ключами, паролей CA или других секретов в отчёте или публичном репозитории считается критической ошибкой безопасности.

Отчёт по практической работе должен содержать:

1. Цель работы и краткое описание предметной области в соответствии с выбранным вариантом.
2. Архитектурную схему разработанной сети Hyperledger Fabric с указанием Organizations, Peer, Ordering Service, Fabric CA, MSP, Gateway, Channel, Chaincode и Ledger.
3. Описание разработанного Chaincode, реализованных бизнес-операций, Endorsement Policy и результатов выполнения транзакций с указанием TxID.

4. Результаты анализа безопасности: модель угроз STRIDE, описание выявленной уязвимости бизнес-логики, способ её воспроизведения и реализованные меры защиты.
5. Конфигурацию Private Data Collection и результаты проверки разграничения доступа к конфиденциальным данным для Org1 и Org2.
6. Результаты обновления Chaincode с version 1.0 до version 2.0 и sequence 1 → 2, включая подтверждение сохранности ранее записанных данных и работоспособности новой функциональности.
7. Результаты исследования SmartBFT при контролируемом отказе одного Orderer и итоговый вывод по работе.

Основные этапы должны подтверждаться командами, результатами их выполнения и скриншотами. Закрытые ключи, пароли, сертификаты с приватными ключами и другие секретные данные в отчёт включать запрещается.

Контрольные вопросы для защиты

1. Объясните полный жизненный цикл транзакции Hyperledger Fabric по модели Execute → Order → Validate → Commit. На каких этапах транзакция может быть признана недействительной и какую роль выполняет MVCC?
2. В чём различие между Endorsement Policy и механизмом авторизации внутри Chaincode? Может ли транзакция удовлетворять Endorsement Policy, но при этом нарушать бизнес-логику приложения? Обоснуйте ответ.
3. В чём различие между Channel **и** Private Data Collection с точки зрения изоляции участников, хранения данных, Ledger и политик доступа? В каких случаях целесообразно применять каждый механизм?
4. Объясните назначение параметров version, sequence и package ID в жизненном цикле Chaincode. Почему при обновлении определения Chaincode необходимо увеличивать sequence и как обеспечивается сохранность ранее записанного World State?
5. В чём принципиальное различие между Raft CFT и SmartBFT BFT? Почему остановка одного Orderer позволяет проверить устойчивость к crash failure, но не является полноценным доказательством устойчивости сети к Byzantine behavior?

Официальные источники

https://hyperledger-fabric.readthedocs.io/en/latest/test_network.html?utm_source - официальное руководство по развёртыванию тестовой сети, созданию Channel, запуску Chaincode и работе с BFT Ordering Service.

https://hyperledger-fabric.readthedocs.io/en/latest/chaincode_lifecycle.html?utm_source - официальный материал по жизненному циклу Chaincode: package, install, approve, commit, version, sequence и endorsement policy.

https://hyperledger-fabric.readthedocs.io/en/latest/private_data_tutorial.html?utm_source - официальное руководство по созданию и использованию Private Data Collections, разграничению доступа и работе с конфиденциальными данными.

https://hyperledger-fabric.readthedocs.io/en/latest/upgrade.html- обновление